ARCHITECTURE & SYSTEM MANUAL

NeuraMorphix Recruitment 2026

Complete Technical Documentation for Frontend, Backend, Database & Email Integration

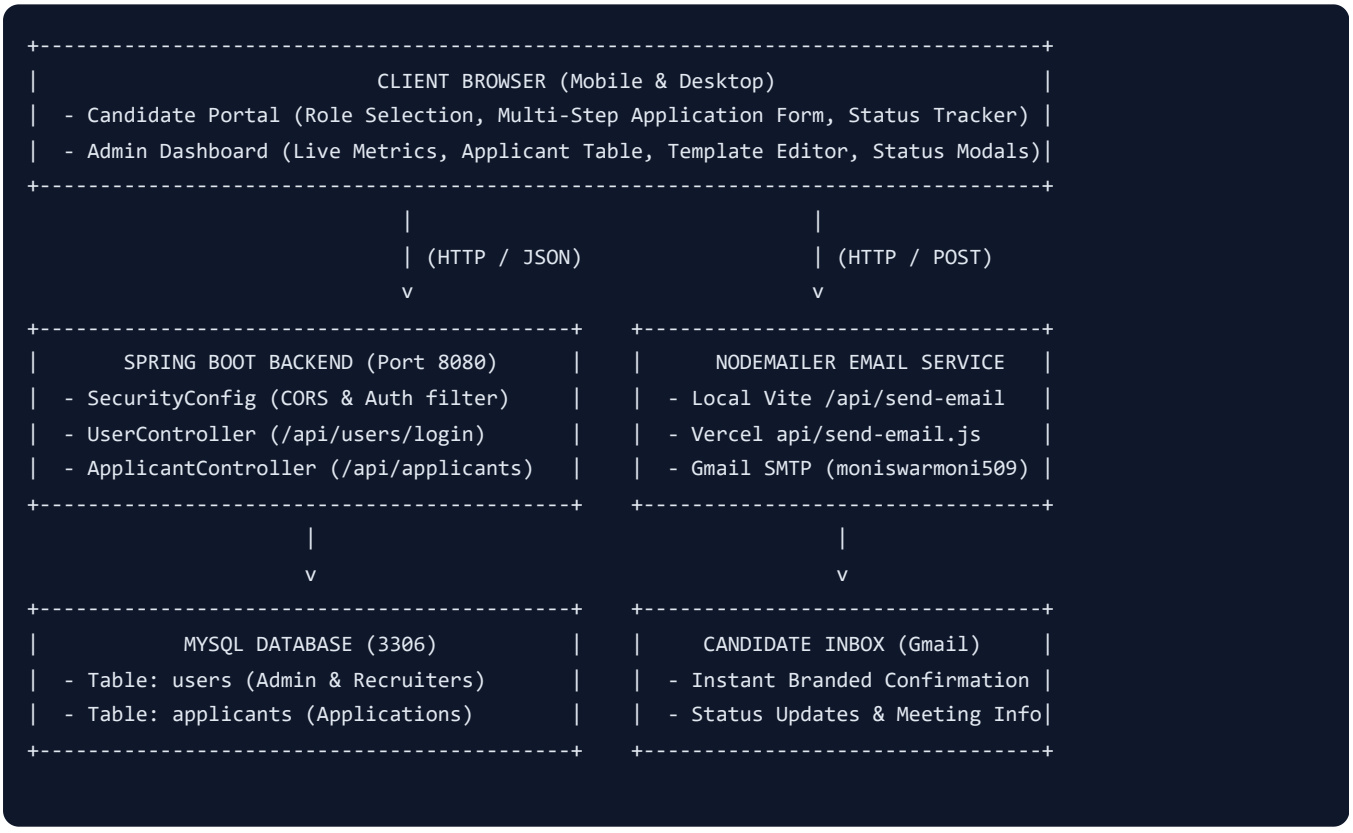
Author: NeuraMorphix Team **Version:** 1.0.0 **Date:** September 2026

1. Executive Summary & System Architecture

The **NeuraMorphix Recruitment Portal** is a modern, full-stack recruitment automation platform engineered for managing candidate applications across 10 specialized technical and creative teams. The system incorporates real-time status tracking, responsive candidate application forms, an administrative command center with analytics, automated Nodemailer-powered email workflows via Gmail SMTP, and a Spring Boot + MySQL relational backend.

> **System Highlights:**
> - **Frontend:** React 19 + TypeScript + Vite + Tailwind CSS v4 + Canvas Confetti
> - **Backend:** Java 25 + Spring Boot 4.1.1 + Spring Security + Spring Data JPA
> - **Database:** MySQL 8.0 / 9.0 (Relational Database) + LocalStorage Cache
> - **Email Delivery:** Nodemailer Gmail SMTP + Vercel Serverless Function
> - **Deployment & CI/CD:** GitHub (`moneswar99/neurapro`) + Vercel SPA Routing

System Architecture Diagram



2. Frontend Architecture & Component Structure

The frontend is written in TypeScript using React 19 and compiled with Vite. It features a responsive layout designed for seamless operation on both desktop and mobile screens.

Key Frontend Components

Component Name	File Path	Purpose & Responsibilities
ApplicationForm	`src/components/ApplicationForm.tsx`	Multi-step application form with college autocomplete, 10-digit phone verification, skill selectors, and submission pipeline.
AdminDashboard	`src/components/AdminDashboard.tsx`	Admin portal for reviewing candidates, updating applicant status (Shortlist, Interview, Accept, Decline), and viewing recruitment analytics.
RoleSelectionSection	`src/components/RoleSelectionSection.tsx`	Interactive grid displaying the 10 NeuraMorphix teams with primary & optional secondary preference selection.
StatusTracker	`src/components/StatusTracker.tsx`	Public applicant tracker allowing candidates to query their Application ID (e.g. `NM-2026-XXXXX`) for status timeline.
EmailInboxDrawer	`src/components/EmailInboxDrawer.tsx`	Live interactive simulated inbox drawer showing sent email previews and delivery statuses in real-time.
Header & Footer	`src/components/Header.tsx`, `Footer.tsx`	Branded navigation bars, mobile bottom navigation tabs, recruitment timer, and portal status pills.

Frontend Services

- **`DatabaseService` (`src/services/db.ts`):** Handles local database state, seed data, initial teams, and synchronizes with `localStorage` for offline support.
- **`EmailService` (`src/services/email.ts`):** Replaces email template variables (e.g. `{{name}}`, `{{application_id}}`) and dispatches requests to `/api/send-email`.
- **`BackendApiService` (`src/services/api.ts`):** Connects frontend components to the Spring Boot REST API for applicant creation and admin login with local fallback.

3. Spring Boot Backend & MySQL Database

The backend is built with **Java 25** and **Spring Boot 4.1.1**. It exposes RESTful API endpoints for user authentication and candidate management backed by Hibernate/JPA.

REST API Endpoints

Method	Endpoint	Description	Request Body
POST	`/api/users/register`	Registers a new recruiter / admin user	`{ "name": "", "email": "", "password": "", "role": "admin" }`
POST	`/api/users/login`	Authenticates admin user and returns user profile	`{ "email": "", "password": "" }`
GET	`/api/applicants`	Retrieves all recruitment applications	*None*
GET	`/api/applicants/{appId}`	Retrieves single applicant by Application ID	*None*
POST	`/api/applicants`	Saves a new recruitment application	`ApplicantEntity` JSON payload
PUT	`/api/applicants/{appId}/status`	Updates applicant status, team, interview details	`{ "status": "Shortlisted", ... }`

Database Schema & Tables

1. Table: `users`

Column Name	Data Type	Constraints	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
`name`	VARCHAR(255)	NOT NULL	Admin / Recruiter full name
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
`password`	VARCHAR(255)	NOT NULL	Authentication password
`role`	VARCHAR(50)	DEFAULT 'admin'	Role permissions

2. Table: `applicants`

Column Name	Data Type	Constraints	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique database ID
`application_id`	VARCHAR(50)	UNIQUE, NOT NULL	Public ID (e.g. NM-2026-48921)

Column Name	Data Type	Constraints	Description
`full_name`	VARCHAR(255)	NOT NULL	Applicant candidate name
`email`	VARCHAR(255)	NOT NULL	Applicant email address
`phone`	VARCHAR(50)	NOT NULL	10-digit mobile number
`college`	VARCHAR(255)	NOT NULL	College / University
`department`	VARCHAR(255)	NOT NULL	Department / Major
`year_of_study`	VARCHAR(50)	NOT NULL	Year of study (1st, 2nd, etc.)
`first_preference`	VARCHAR(255)	NOT NULL	1st choice team name
`second_preference`	VARCHAR(255)	NULLABLE	2nd choice team name
`status`	VARCHAR(50)	DEFAULT 'Application Received'	Recruitment status
`created_at`	DATETIME	NOT NULL	Submission timestamp

4. Automated Email Delivery System

The recruitment platform uses **Nodemailer** configured with Gmail SMTP to dispatch immediate, beautifully styled HTML recruitment notifications to applicants upon form submission and status progression.

Email Configuration Parameters

- **SMTP Provider:** Gmail SMTP (`smtp.gmail.com`)
- **System Email:** `moniswarmoni509@gmail.com`
- **Security:** 16-character Google App Password (`rzlcebjxhpgbumqb`)
- **Supported Environments:**
  - Local Dev Server: Middleware endpoint inside `vite.config.ts`
  - Standalone Server: Express server in `server.js`
  - Production / Vercel: Serverless Function in `api/send-email.js`

Automated Email Stages

Event Type	Subject Line Format	Trigger Point
`application_received`	NeuraMorphix Recruitment — Application Received (`{{application_id}}`)	Triggered immediately when candidate submits application.
`shortlisted`	Congratulations! Shortlisted for NeuraMorphix 2026	Triggered by Admin clicking "Shortlist".
`interview`	Interview Scheduled — NeuraMorphix Recruitment 2026	Triggered by Admin scheduling interview with Google Meet link.
`info_requested`	Action Required: Information Requested for NeuraMorphix	Triggered when recruiter requests GitHub / Portfolio.
`accepted`	Welcome to NeuraMorphix! Application Accepted	Triggered when candidate is accepted into assigned team.
`declined`	NeuraMorphix Recruitment 2026 — Application Update	Polite notification when role capacity is reached.

5. Execution & Deployment Guide

1. Running the Frontend

```
# Navigate to project directory
cd c:\Users\mones\OneDrive\Desktop\NeuraMorphix

# Start Vite dev server with network exposure
npm run dev

# Accessible at:
# Local:   http://localhost:5173/
# Mobile:  http://<your-wifi-ip>:5173/
```

2. Running the Spring Boot Backend

```
# Navigate to backend directory
cd c:\Users\mones\OneDrive\Desktop\NeuraMorphix\backend

# Run Maven Spring Boot target
.\mvnw.cmd spring-boot:run

# Backend active at: http://localhost:8080/
```

3. Viewing Application Data & Candidate Counts

- **In Admin Portal:** Open `http://localhost:5173/` → Click **Admin Login** → Enter `moni@neuramorphix.com` / `admin123`. The **Analytics** and **Applicants** tabs show live statistics.
- **In MySQL:** Run `SELECT COUNT(*) FROM applicants;` inside database `recruitment_db`.
- **In Browser Storage:** Inspect element (F12) → Local Storage → `neuramorphix_applicants_v1`.

4. Production Build & Deployment

```
# Build production bundle
npm run build

# Push to GitHub
git add .
git commit -m "Deploy latest build"
git push origin main
```

> **Conclusion:**
> The NeuraMorphix Recruitment platform is complete, tested, and fully configured for production deployment

> across desktop and mobile devices.